



Institut Supérieur d'Informatique
et de Multimédia de Gabès

Programmation Orientée Objet 2

JAVA

Sofiane HACHICHA

sofiane.hachicha@isimg.tn

CPI2

2025/2026



Institut Supérieur d'Informatique
et de Multimédia de Gabes

P00 : **Java avancé**



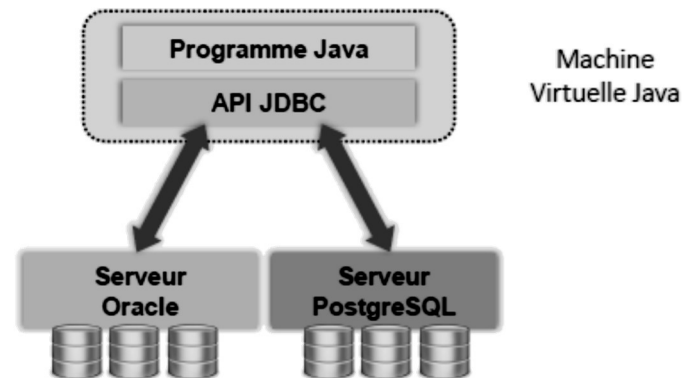
Chapitre

5

L'accès aux bases de données

● JDBC : Java Data Base Connectivity

- API java standard qui permet un accès homogène à des bases de données depuis un programme Java au travers du langage SQL.



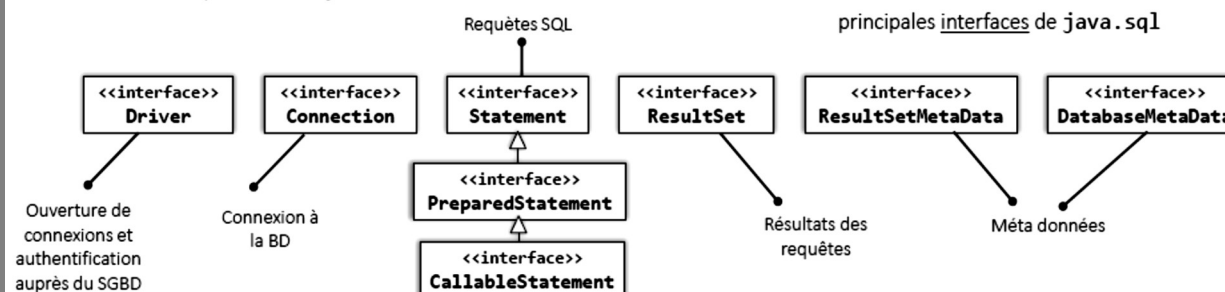
● L'API JDBC est indépendante du SGBD.

- Un changement de SGBD ne doit pas impacter le code applicatif.



● L'API JDBC est basée sur un ensemble d'**interfaces** définies dans le package **java.sql** qui établissent un protocole de communication entre le programme Java client et le serveur de base de données permettant :

- Ouverture/fermeture de connexions à une BD.
- Exécution de requêtes SQL.
- Exploitation des résultats.
- Correspondance types SQL-types Java.
- Accès au méta-données.





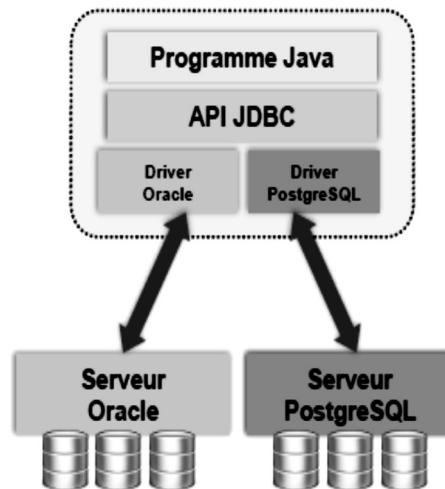
5.2 Etapes d'utilisation de JDBC



- 1. Chargement du pilote JDBC : facultatif depuis JDBC 4.0 (Java 6)**
- 2. Définition de l'URL de connexion**
- 3. Ouverture d'une connexion avec la BD**
- 4. Création d'une zone de description de requête (Statement, PreparedStatement, CallableStatement)**
- 5. Exécution de la requête**
- 6. Récupération et traitement des résultats**
- 7. Fermeture des différents espaces**



- Le code applicatif est basé sur les interfaces du JDBC
- Pour accéder à un SGBD, il est nécessaire de disposer des classes implémentant ces interfaces.
 - Elles dépendent du SGBD adressé.
 - L'ensemble de ces classes pour un SGBD donné est appelé **pilote (driver) JDBC**



- » Il existe des pilotes pour tous les SGBD importants du marché (Oracle, MySQL, PostgreSQL, DB2, SQLite, etc.).
- » JDBC spécifie uniquement l'API que ces pilotes doivent respecter.
- » L'implémentation des drivers est totalement libre.



5- JDBC

5.2.1 Pilotes JDBC

MySQL Connector/J 8.0.23

P00 : Java avancé



Sofiane HACHICHA
ISIMG 2025 - 2026

```
mysql-connector-java-8.0.23.jar
├── com.mysql.cj
│   ├── com.mysql.cj.admin
│   ├── com.mysql.cj.conf
│   ├── com.mysql.cj.conf.url
│   ├── com.mysql.cj.configurations
│   ├── com.mysql.cj.exceptions
│   ├── com.mysql.cj.interceptors
│   └── com.mysql.cj.jdbc
│       ├── AbandonedConnectionCleanupThread.class
│       ├── Blob.class
│       ├── BlobFromLocator.class
│       ├── CallableStatement.class
│       ├── CallableStatementWrapper.class
│       ├── ClientInfoProvider.class
│       ├── ClientInfoProviderSP.class
│       ├── ClientPreparedStatement.class
│       ├── Clob.class
│       ├── CommentClientInfoProvider.class
│       ├── ConnectionGroup.class
│       ├── ConnectionGroupManager.class
│       ├── ConnectionImpl.class
│       ├── ConnectionWrapper.class
│       ├── DatabaseMetaData.class
│       ├── DatabaseMetaDataUsingInfoSchema.class
│       ├── Driver.class
│       ├── EscapeProcessor.class
│       ├── EscapeProcessorResult.class
│       ├── IterateBlock.class
│       ├── JdbcConnection.class
│       ├── JdbcPreparedStatement.class
│       ├── JdbcPropertySet.class
│       ├── JdbcPropertySetImpl.class
│       └── JdbcStatement.class
```

com.mysql.cj.jdbc.Driver.class - IntroIPA/lib/mysql-connector-java-8.0.23.jar

- » Les interfaces définissent une **abstraction** du pilote (driver) de la base de données. Chaque fournisseur propose sa propre implémentation de ces interfaces.
- » Les classes d'implémentation du driver jdbc sont fournies dans une archive (fichier **.jar**) qu'il faut intégrer (sans la décompresser) au niveau du **CLASSPATH** de l'application durant la compilation/exécution.
- » Au niveau du programme d'application **on ne travaille qu'avec** les abstractions (interfaces) sans se soucier des classes effectives d'implémentation.





● La liste des drivers est disponible :
<https://www.oracle.com/java/technologies/industry-support.html>

● sur les sites des fournisseurs de BD

● SQLite :

<https://repo1.maven.org/maven2/org/xerial/sqlite-jdbc/>

● Oracle :

<https://www.oracle.com/database/technologies/appdev/jdbc-downloads.html>

● PostgreSQL :

<https://jdbc.postgresql.org/download/>

MySQL :

<https://dev.mysql.com/downloads/connector/j/>

● ...





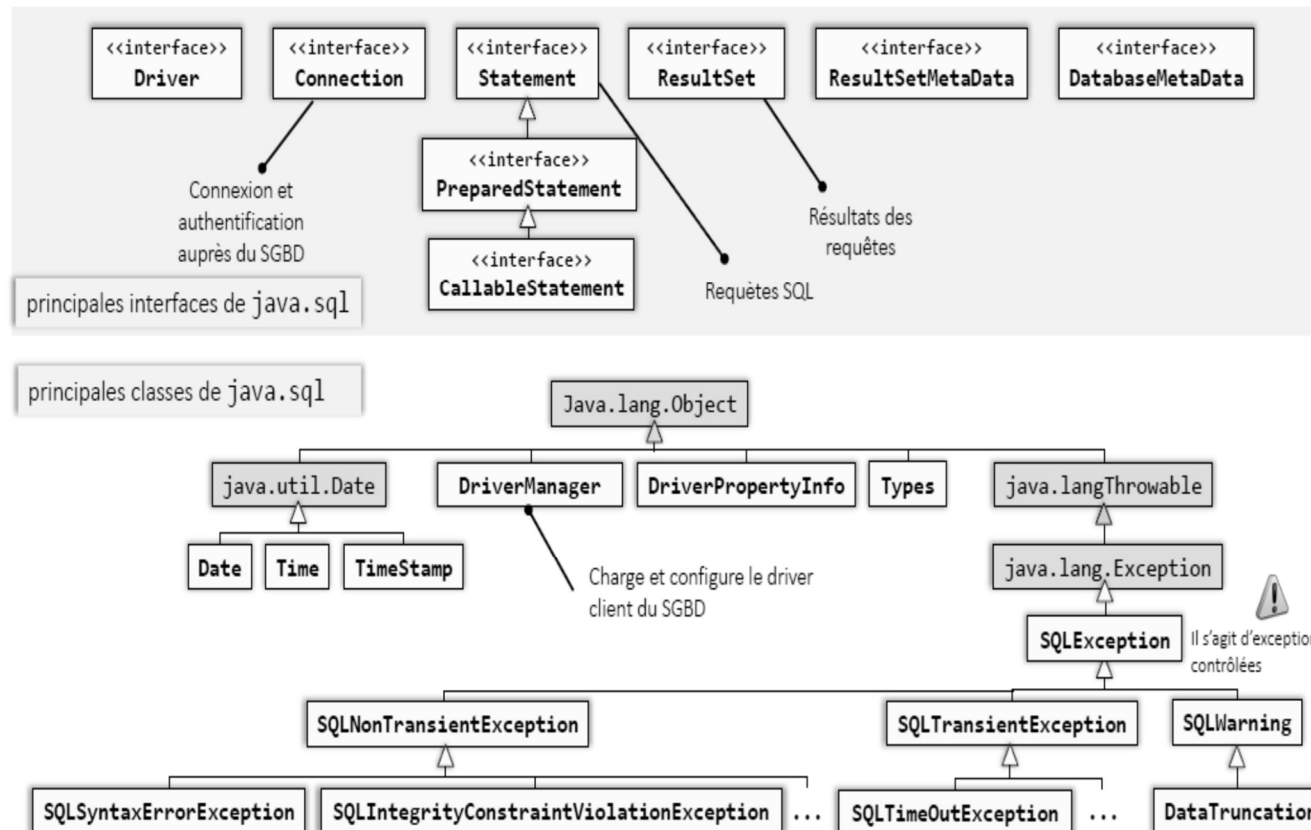
● Différentes versions :

- JDBC 1.0 Core API (JDK 1.1) : paquetage java.sql
 - supporte le standard SQL-2
- JDBC 2.0 (JDK 1.2) : paquetages java.sql javax.sql
 - support certaines fonctionnalités de SQL-3
- JDBC 3.0 (JDK 1.4) : paquetages java.sql javax.sql javax.sql.rowset
 - support d'autres fonctionnalités de SQL-3
 - nouveau support pour la gestion des ResultSet
- JDBC 4.0 (JDK 6)
 - facilité d'écriture à travers les annotations



- JDBC 4.1 (JDK 7)
 - Try-with-resources pour la fermeture des ressources telles que les connexions JDBC, les requêtes et les résultats.
 - RowSet 1.1 : une extension de ResultSet qui offre une approche plus flexible et orientée objet pour manipuler les résultats de requêtes JDBC.
- JDBC 4.2 (JDK 8)
 - nouveaux supports pour les types de données et des changements dans les interfaces existantes.
 - RowSet 1.2
- JDBC 4.3 (JDK 9)
 - Introduction des méthodes beginRequest() et endRequest() dans l'interface Connection permettant aux applications de gérer plus efficacement les requêtes multiples et les transactions JDBC.

5.2.3 Classes et interfaces de JDBC



5.2.3 Classes et interfaces de JDBC



● Fonctionnement général de JDBC :

1. Etablir une connexion à la base de données : **DriverManager** permet de créer des objets **Connection** (objets d'une classe qui implémente l'interface Connection)

Connection c = DriverManager.getConnection(...);

2. Créer un objet de transport qui permettra d'envoyer des requêtes SQL à la BD : L'objet **Connection** permet de créer des objets **Statement** encapsulant des requêtes SQL

Statement stmt = c.createStatement(...);

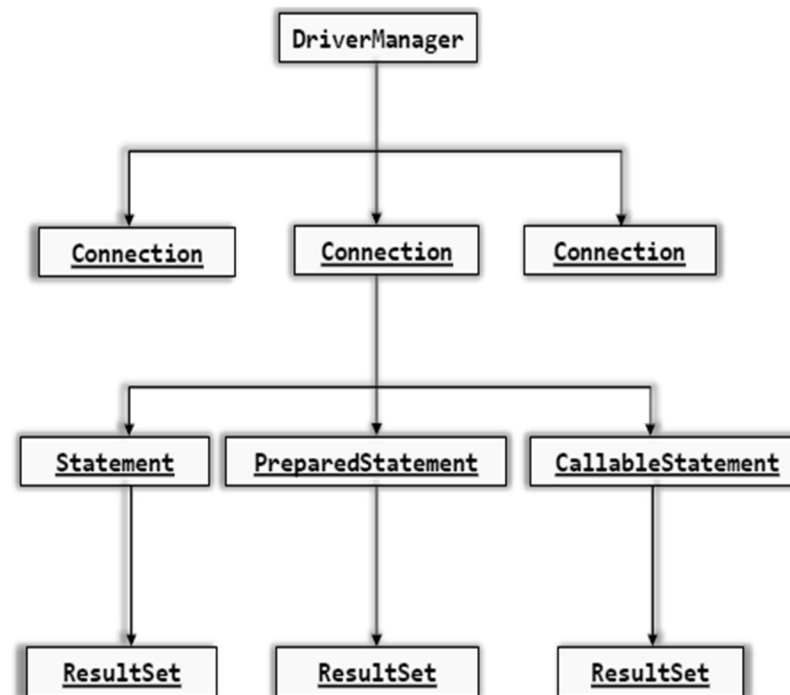
3. Exécuter la requête sur la BD et récupérer ses résultats : L'objet **Statement** permet de créer des objets **ResultSet** encapsulant le résultat d'une requête

ResultSet rs = stmt.executeQuery(...);

5.2.3 Classes et interfaces de JDBC

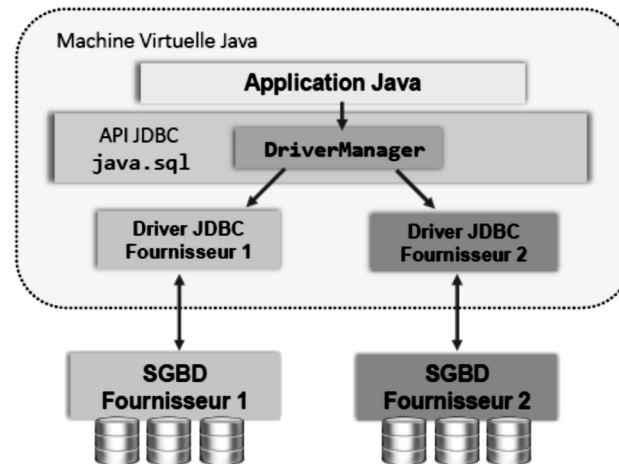
4. Exploiter les résultats de la requête : L'objet **ResultSet** permet de parcourir le résultat de la requête et de récupérer les données

while (rs.next()) { ... }





- **DriverManager** : classe Java à laquelle s'adresse le code de l'application cliente pour initier JDBC.



» Le Driver Manager permet de charger et configurer les pilotes JDBC nécessaires à l'application.



5.3.1 Chargement du driver



- Avant de pouvoir être utilisé, le driver doit être enregistré auprès du DriverManager de jdbc.

```
DriverManager.registerDriver(new oracle.jdbc.OracleDriver());
```

- Il est **préférable** d'exploiter les possibilités de **chargement dynamique** de classes de JAVA

- Utiliser la méthode **forName** de la classe **Class** avec en paramètre le nom complet de la classe du driver.

```
try {
    Class.forName("org.sqlite.JDBC");
    Class.forName("oracle.jdbc.OracleDriver") .newInstance();
}
catch (ClassNotFoundException e) {
    ...
}
```

Chargement d'un pilote SQLite

Chargement d'un pilote Oracle

- Afin qu'il soit possible de changer le driver sans modifier l'application : Stocker le nom du driver dans un fichier de configuration (properties file).





5.4 Connexion à la base



● Ouverture de la connexion :

```
Connection conn = DriverManager.getConnection(url,user, password);
```

● Identification de la BD via une **URL** de la forme générale :

jdbc : driver : base

utilisation de JDBC le driver ou le type de SGBDR identification de la base

- La forme exacte de l'**URL dépend de la BD**. Chaque BD nécessite des informations spécifiques pour établir la connexion.

- Par exemple pour le driver Oracle JDBC-Thin :

jdbc : oracle:thin : @serveur:port:base

nom IP du	numéro de port	nom de la
serveur	socket à utiliser	base



● Exemple d'une connexion à une BD :

- Paramètres de connexion définis dans un fichier propriétés (properties file) :

fichier texte de couples nomprop=valeur

Il faut que les classes du driver soient déjà dans le CLASSPATH

```
conf.properties
1# C'est un commentaire
2jdbc.driver=com.mysql.cj.jdbc.Driver
3jdbc.url=jdbc:mysql://localhost:3306/webstore
4jdbc.user=root
5jdbc.password=*****
```

```
public static Connection ouvrirConnexion(String driverClass, String dbURL, String userId, String passwd)
    throws ClassNotFoundException, SQLException {
    Class.forName(driverClass);
    return DriverManager.getConnection(dbURL, userId, passwd);
}
```

```
Properties props = new Properties();
props.load(new FileInputStream( "conf.properties" ));
```

```
String driverClass = props.getProperty("jdbc.driver");
String dbURL = props.getProperty("jdbc.url");
String userId = props.getProperty("jdbc.user");
String passwd = props.getProperty("jdbc.password");
```

Chargement
des propriétés

```
Connection conn = ouvrirConnexion (driverClass , dbURL , userId , passwd);
```

```
//...
```

```
conn.close();
```



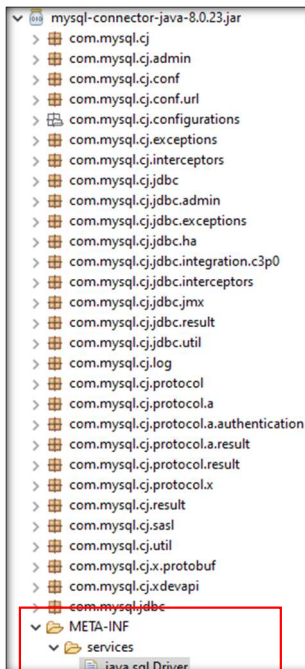
5.4 Connexion à la base



● A partir de **JDBC 4.0**, il n'est plus besoin de charger explicitement le pilote JDBC

```
public static Connection ouvrirConnexion(String driverClass, String dbURL,  
String userId, String passwd) throws ClassNotFoundException, SQLException  
Class.forName(driverClass);  
return DriverManager.getConnection(dbURL, userId, passwd);  
}
```

inutile avec Pilote JDBC4.0+. Le chargement des pilotes est automatique



- » Utilisation du mécanisme Java (Java 6+) pour les fournisseurs de services
 - » service défini par une interface par exemple pour un pilote JDBC : **java.sql.Driver**
 - » si un fichier **.jar** contient un code d'implémentation du service, son répertoire **META-INF/services** contient un fichier de nom identique au service et dont le contenu est le nom de la classe d'implémentation du service
- » Le **DriverManager** parcourt tous les **.jar** présents dans le **classpath** et peut charger **automatiquement** tous les drivers compatibles JDBC 4.0+ .



5.4 Connexion à la base



- **Pour économiser les ressources, il est parfois préférable de créer un seul objet de connexion.**
 - Le patron de conception de création **Singleton** a pour but de contrôler la création d'objets, en limitant le nombre d'objets en un seul. Il permet de disposer d'une instance unique d'un objet.
- **Le patron Singleton contrôle souvent l'accès aux ressources partagées, telles que les connexions de bases de données, les sockets réseau, les gestionnaires de fichiers, etc.**
 - Si on a une licence pour une seule connexion à la base de données, le patron Singleton assure qu'une seule connexion est établie.
- **Ainsi, il faut garantir qu'une seule instance de Connection existe dans l'application.**
 - Le patron Singleton repose sur un **constructeur privé** associé à **une méthode retournant l'instance** créée dans la classe elle-même.





```
import java.sql.*;
public class ExempleConnection{
    private String url = "...";
    private String user = "..."; private String passwd = "...";
    //Objet Connection
    private static Connection connect;
    //Constructeur privé
    private ExempleConnection(){
        try {
            connect = DriverManager.getConnection(url, user, passwd);
        } catch (SQLException e)
        { e.printStackTrace(); }
    }
    //Méthode qui retourne l'objet connection et le crée s'il n'existe pas
    public static Connection getInstance(){
        if(connect == null){
            new ExempleConnection();
        }
        return connect;
    }
}
```



5.4 Connexion à la base



```
import java.sql.*;
public class ExempleConnection{
//Objet Connection
    private static Connection connect;
    public static Connection getInstance() throws FileNotFoundException,
    IOException{
        Properties p = new Properties();
        p.load(new FileInputStream("conf.properties"));
        if(connect == null){
            try {
                connect = DriverManager.getConnection(p.getProperty("jdbc.url"),
                    p.getProperty("jdbc.user"),p.getProperty("jdbc.password"));
            }
            catch (SQLException e) {
                e.printStackTrace();
            }
        }
        return connect;
    }
}
```

5.4.1 Fermeture de la connexion



- **Une application peut maintenir des connexions multiples**
 - le nombre limite de connexions est fixé par le SGBD lui-même (de quelques dizaines à des milliers).
- **Quand une connexion n'a plus d'utilité, on doit prendre soin de la fermer explicitement.**
 - Libération de mémoire et surtout des ressources de la base de données détenues par la connexion

```
try {
    Connection conn = DriverManager.getConnection(url, user, passwd);
    ...
    // utilisation de la connexion pour dialoguer avec la BD
    ...
    // fermeture de la connexion
    conn.close();
} catch (SQLException e) {
    ...
}
```

Erreur SQL lors du dialogue avec la BD

L'instruction close n'est pas exécutée.
La connexion reste ouverte !

Comment garantir la fermeture de la connexion ?



5.4.1 Fermeture de la connexion



● Pour garantir fermeture de la connexion, on doit utiliser une clause finally. Depuis JDBC 4.1 et Java 7, l'utilisation du try-with-resources est recommandée pour garantir la fermeture automatique des ressources, y compris les connexions à la base de données.

Pour que conn soit connue dans le bloc finally

● `Connection conn = null;` ●

Le compilateur impose d'initialiser conn car sinon message d'erreur : conn peut ne pas avoir été initialisée

```
try {
    conn = DriverManager.getConnection(url, user, passwd);
    ...
    // utilisation de la connexion pour dialoguer avec la BD
    ...
}
```

```
catch (SQLException e) {
    ...
}
```

finally {

```
try {
    if (conn != null)
        conn.close();
} catch (SQLException e){
    e.printStackTrace();
}
```

close peut provoquer une SQLException

utilisation d'un try avec ressources (Java7+) au lieu de la clause finally

```
try (Connection conn =
    DriverManager.getConnection(url, user, passwd)) {
    ...
    // utilisation de la connexion pour dialoguer avec la BD
    ...
} catch (SQLException e) {
    ...
}
```

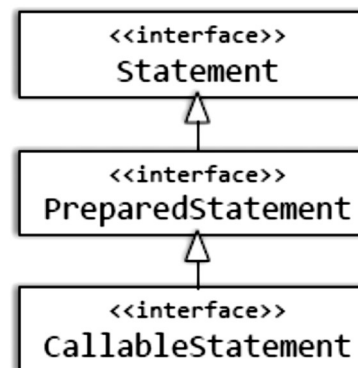
Les ressources sont automatiquement fermées à la fin de l'instruction try



5.5 Préparer/exécuter une requête



- Une fois la **Connection** créée, on peut l'utiliser pour créer et exécuter des requêtes (statements) SQL.
- 3 types d'objets statement (interfaces) :
 - **Statement** : requêtes simples (SQL statique).
 - **PreparedStatement** : requêtes **précompilées** (SQL dynamique si supporté par SGBD) qui peuvent améliorer les performances et la sécurité.
 - **CallableStatement** : encapsule des procédures SQL stockées dans le SGBD.



5.5 Préparer/exécuter une requête



● 3 formes d'exécutions (méthodes) selon le type de la requête :

- requêtes de consultation (SELECT)
 - ResultSet **executeQuery**(String sql) → un objet ResultSet pour accéder à la table résultat.
- requêtes de mise à jour (INSERT, UPDATE, DELETE, CREATE TABLE et DROP TABLE)
 - int **executeUpdate**(String sql) → un entier (le nombre d'enregistrements affectés par la requête)
- requêtes ouvertes (on ne connaît pas a priori la nature de la requête)
 - boolean **execute**(String sql) → true si la requête est de type SELECT, false si la requête est de mise à jour

5.5 Préparer/exécuter une requête



● Création d'un statement :

- A partir de de l'objet **Connection**, on récupère l'objet **Statement** associé :

- `Statement stmt = conn.createStatement();`
- `PreparedStatement stmt = conn.prepareStatement(sql);`
- `CallableStatement stmt = conn.prepareCall(sql);`

● Exécution de la requête :

```
String myQuery = "SELECT prenom, nom, email." +  
                "FROM employe." +  
                "WHERE (nom='Dupont') AND (email IS NOT NULL)." +  
                "ORDER BY nom";
```

la requête est une chaîne SQL qui doit être valide

⚠ espaces nécessaires si
concaténation de chaînes

```
ResultSet rs = stmt.executeQuery(myQuery) ;
```

référence vers un objet **ResultSet**
qui permettra d'exploiter les résultats

appel de méthode `executeQuery`
car requête de type **SELECT**



5.6 Lecture des résultats



- **executeQuery()** renvoie un objet de classe **ResultSet**
- Le **parcours des rangées ResultSet** est **itératif ligne par ligne**
 - Curseur de lecture placé **avant la première ligne** à la création du **ResultSet**.
 - **boolean next()** permet d'avancer à la ligne suivante
→ false si pas de ligne suivante
 - Il faut appeler une fois **next()** avant d'accéder à la première ligne de résultat
- Les **colonnes du ResultSet** sont **référéncées par leur nom** ou par leur **numéro**
 - L'accès aux valeurs des colonnes se fait par les méthodes **getXXX(String nomCol)** ou **getXXX(int numCol)** où XXX dépend du type de la colonne dans la table SQL.



5.6 Lecture des résultats



```
java.sql.Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT colA, colB, colC FROM Table1");

while (rs.next()) {
    int entier = rs.getInt("colA");          // rs.getInt(1);
    String chaine = rs.getString("colB");    // rs.getString(2);
    byte bytes[] = rs.getBytes("colC");     // rs.getBytes(3);
    System.out.println("ROW = " + entier + " " + chaine + " " + bytes[0]);
}
```

Attention ! En SQL les
numéros de colonnes
débutent à 1

- Pour chaque méthode **getXXX()**, le driver JDBC doit effectuer une conversion entre le type de données de la base de données et le type Java correspondant



5.6.1 Equivalences des types Java-SQL

Type SQL	Méthode	Type Java
CHAR	getString	String
VARCHAR	getString	String
NUMERIC	getBigDecimal	java.Math.BigDecimal
DECIMAL	getBigDecimal	java.Math.BigDecimal
BIT	getBoolean	boolean <i>Boolean</i>
TINYINT	getByte	byte <i>Integer</i>
SMALLINT	getShort	short <i>Integer</i>
INTEGER	getInt	int <i>Integer</i>
BIGINT	getLong	long <i>Long</i>
REAL	getFloat	float <i>Float</i>
FLOAT	getDouble	double <i>Double</i>
DOUBLE	getDouble	double <i>Double</i>
DATE	getDate	java.sql.Date
TIME	getTime	java.sql.Time
TIME STAMP	getTimestamp	java.sql.Timestamp

getString() peut être appelée sur
n'importe quel type de valeur

getObject() peut retourner n'importe
quel type de donnée « packagé » dans un
objet java (wrapper object)

Si une conversion de données invalide est
effectuée (par exemple DATE -> int),
une SQLException est lancée

5.6.2 Traitement des valeurs nulles



- **Que se passe-t-il si une méthode `getXXX()` de `ResultSet` est appliquée à une valeur `NULL SQL` ?**
 - Conversion automatique vers une valeur selon le type retourné par `getXXX()`
 - **null** si `getXXX()` retourne un type objet (ex : `getString()`, `getDate()`,...)
 - ● si `getXXX()` retourne un type numérique (ex : `getInt()`, `getDouble()`,...)
 - **false** pour `getBoolean()`
- **Pour repérer les valeurs `NULL` de la base, il faut appeler la méthode **`wasNull()`** de `ResultSet` :**
 - Renvoie `true` si l'on vient de lire une valeur `NULL`, `false` sinon.

5.6.2 Traitement des valeurs nulles



● Exemple :

```
...  
  
Statement stmt = conn.createStatement();  
  
ResultSet rs = stmt.executeQuery( "SELECT nomE, commE FROM EMPLOYE");  
  
while (rs.next()) {  
    String nom = rs.getString(1);  
    Double commission = rs.getDouble(2);  
    if (rs.wasNull())  
        System.out.println(nom + ": pas de comm");  
    else  
        System.out.println(nom + " a " + commission + "dinars de  
                             commission");  
}
```



5.6.3 Requête précompilée : PreparedStatement



- L'interface **PreparedStatement** définit les méthodes pour un objet qui va encapsuler une requête précompilée.
 - Ce type de requête est particulièrement adapté pour une exécution répétée d'une même requête avec des paramètres différents.
- Lors de l'utilisation d'un objet de type **PreparedStatement**, la requête est envoyée au moteur de la base de données pour que celui-ci prépare son exécution.
- Un objet qui implémente l'interface **PreparedStatement** est obtenu en utilisant la méthode **prepareStatement()** d'un objet de type **Connection**.
 - Cette méthode attend en paramètre une chaîne de caractères contenant la requête SQL. Dans cette chaîne, chaque paramètre est représenté par un caractère ?.

```
PreparedStatement ps = conn.prepareStatement("SELECT *  
FROM EMPLOYE WHERE nomE= ? ");
```

5.6.3 Requête précompilée : PreparedStatement



- L'interface **PreparedStatement** propose des méthodes **setXXX()** permettant de fournir les valeurs de chaque paramètre défini dans la requête.

```
ps.setXXX(int parameterIndex, XXX x)
```

- Le premier paramètre de ces méthodes précise le numéro du paramètre dont la méthode va fournir la valeur. Le second paramètre précise cette valeur.

- **setNull(n, sqlType)** assigne une valeur NULL SQL au paramètre d'indice n.

- Invocation par **executeQuery()** (ou **executeUpdate()**) sans paramètres.

```
ps.setString(1, "user");
```

```
ResultSet rs= ps.executeQuery();
```

```
// ... Exploitation des résultats
```

phase identique à celle utilisée par Statement

5.6.4 Procédure stockée : CallableStatement



- Une procédure stockée permet de fournir la même fonctionnalité à plusieurs utilisateurs. Elle est stockée dans la BD coté serveur.
- Utilisation des procédures stockées depuis JDBC via interface **CallableStatement**
 - Syntaxe unifiée indépendante de la manière dont celles-ci sont gérées par n'importe quel SGBD.
 - Utilisation possible de la valeur de retour.
 - Gestion des paramètres IN, OUT, INOUT.
- Préparation de l'appel
 - Appel avec valeur de retour et avec paramètres

```
CallableStatement proc = conn.prepareCall("{ ? = call maProcedure(?, ?) }");
```
 - Appel sans valeur de retour et avec paramètres

```
CallableStatement proc = conn.prepareCall("{ call maProcedure(?, ?) }");
```



5.6.4 Procédure stockée : CallableStatement



● Préparation des paramètres OUT

```
proc.registerOutParameter(2, Types.INTEGER);
```

2^{ème} paramètre de type OUT

● Passage des paramètre IN

```
proc.setByte(1, 25);
```

1^{er} paramètre de type IN

● Appel à `executeQuery()` (ou `executeUpdate()` ou `execute()`) sans paramètres

```
ResultSet rs = proc.executeQuery();
```

● Exploitation du `ResultSet` est similaire lors de l'utilisation de `Statement` et de `PreparedStatement`.

● Récupération des paramètres OUT

```
int result = proc.getInt(2);
```

5.7 Fermeture des Statement et ResultSet



● Un objet Statement représente une simple (seule) requête SQL.

- Un appel à `executeQuery()` , `executeUpdate()` ou `execute()` ferme implicitement tout `ResultSet` actif associé avec l'objet `Statement`.
- Avant d'exécuter une autre requête avec un même objet `Statement`, il faut être sûr d'avoir exploité les résultats de la requête précédente.

```
Statement stmt = conn.createStatement();

ResultSet rs1 = stmt.executeQuery(myQuery1);
ResultSet rs2 = stmt.executeQuery(myQuery2);

//exploitation des résultats de myQuery1
while (rs1.next() {
    ...
}
//exploitation des résultats de myQuery2
while (rs2.next() {
    ...
}
```

```
Statement stmt = conn.createStatement();

ResultSet rs1 = stmt.executeQuery(myQuery1);
//exploitation des résultats de myQuery1
while (rs1.next() {
    ...
}

ResultSet rs2 = stmt.executeQuery(myQuery2);
//exploitation des résultats de myQuery2
while (rs2.next() {
    ...
}
```

- Si application nécessite d'effectuer plus d'une requête simultanément, il faut créer et utiliser autant d'objets `Statement`.

5.7.1 Fermeture des différents espaces



- **Pour terminer proprement un traitement, il faut fermer tous les objets ouverts depuis le début du programme**
 - sinon le Garbage Collector (GC) s'en occupera mais c'est moins efficace.
- **Chaque objet possède une méthode close() :**

```
rs.close();  
stmt.close();  
conn.close();
```
- **Utiliser simplement le try -with-resources pour ces objets**
 - Ces ressources seront automatiquement fermées à la fin de l'instruction try.



5.8 Exemple d'utilisation de JDBC



● Une entreprise souhaite gérer les informations de ses employés. Elle dispose d'une base de données nommée "entreprise_db" avec une table "employees" ayant les colonnes suivantes : "id"(clé primaire auto-incrémentée, entier), "nom" (chaîne de caractères), "age" (entier) et "salaire" (décimal). Écrivez un programme Java qui permet de :

1. Établir une connexion unique à la base de données "entreprise_db" en utilisant le pattern Singleton et en fournissant un nom d'utilisateur ("user") et un mot de passe ("cpi2") pour accéder à la base de données.
2. Créer la table "employees" avec les colonnes appropriées.
3. Insérer des données dans la table "employees" en utilisant des requêtes préparées. Les informations des employés sont les suivantes :





5.8 Exemple d'utilisation de JDBC



- Employé 1 : nom = "Rami Krid", age = 34, salaire = 2200.500
- Employé 2 : nom = "Sami Feki", age = 28, salaire = 1500.750
- Employé 3 : nom = "Salma Abid", age = 24, salaire = 1200.000

4. Effectuer une requête pour récupérer tous les employés de la table "employees" et afficher leurs informations à la console.
5. Effectuer une requête préparée pour mettre à jour le salaire de l'employé ayant l'id 1 et l'augmenter de 10%.

5.9 Accès aux méta-données



- Permet de découvrir dynamiquement (au moment de l'exécution) des propriétés concernant la base de données ou les résultats de requêtes
 - La méthode **getMetaData()** invoquée à partir d'un objet de type **Connection** permet d'obtenir les méta-données concernant la BD.
 - Elle renvoie un **DatabaseMetaData**.
 - On peut connaître :
 - les éléments SQL supportés par la BD
 - la structure des données de celle-ci (getCatalog, getTables...)
 - La méthode **getMetaData()** invoquée à partir d'un objet de type **ResultSet** permet d'obtenir les méta-données d'un ResultSet.
 - Elle renvoie un **ResultSetMetaData**.



● On peut connaître :

- le nombre de colonnes : getColumnCount()
- le nom d'une colonne : getColumnName(intcol)
- le type d'une colonne : getColumnType(intcol),
- ...

● **Exemple : Lors de l'exécution d'une requête non connue à l'avance par la méthode execute() :**

```

Renvoie true si requête de type Query, false sinon (Update)
String contenant une requête quelconque

if stmt.execute(cmd) {
    ResultSet rs = stmt.getResultSet();
    ...
    //Exploitation du ResultSet
    ...
    //Besoin d'accès aux méta-données du ResultSet
    ...
    rs.close();
}
else
    System.out.println("nombre de lignes modifiées " + stmt.getUpdateCount() );
}

ResultSetMetaData rsmd = rs.getMetaData();
int numberOfColumns = rsmd.getColumnCount();
//affichage du resultat de la requête
System.out.println("Résultats de la requête\n");
while (rs.next()) {
    System.out.println("-----");
    for (int i = 1; i <= numberOfColumns; i++) {
        System.out.print(rsmd.getColumnName(i) + " : " + rs.getString(i) + " ");
    }
}

entier donnant le nombre d'enregistrements modifiés
    
```

Accès au ResultSet produit par la requête

Besoin d'accès aux méta-données du ResultSet

5.10 Gestion des transactions



- **Transaction** : permet de ne valider un ensemble de traitements sur une BD que si ils se sont tous effectués correctement
 - Exemple : transfert d'argent = débiter un compte + créditer un autre compte
- L'interface **Connection** offre des services de gestion des transactions
 - Pour pouvoir rassembler plusieurs traitements dans une transaction et gérer soi-même les transactions, il faut **désactiver le mode auto-commit** qui est par défaut activé : **conn.setAutoCommit(false);**
 - Pour valider la transaction en cours : **conn.commit();**
 - Pour annuler la transaction: **conn.rollback();**

5.10 Gestion des transactions



Exemple

```
...
try {
    conn.setAutoCommit(false);
    stmt=conn.createStatement();
    // exécuter les instructions qui constituent la transaction
    stmt.executeUpdate("update users set nom='user1' where id=200");
    stmt.executeUpdate("insert into users values(201,'test','isimg')");
    ...
    // valider la transaction
    conn.commit();
}
catch (SQLException e) {
    // annuler les opérations de la transaction
    conn.rollback();
}
```



5.10 Gestion des transactions



- API JDBC 3.0 a ajouté la possibilité de définir des points de sauvegarde dans une transaction

```
Statement stmt = conn.createStatement();

conn.setAutoCommit(false);
...
int rows = stmt.executeUpdate("INSERT INTO TAB1 ... ");
// set savepoint
Savepoint svpt1 = conn.setSavepoint("SAVEPOINT_1");
rows = stmt.executeUpdate("INSERT INTO TAB1 ... ");
...
conn.rollback(svpt1);    Annule toutes les opérations effectuées
                        depuis le point de sauvegarde
...
conn.commit();
```

- Pour retirer un point de sauvegarde

```
conn.releaseSavePoint("SAVEPOINT_1");
```

- SAVEPOINT_1 ne peut plus être utilisé dans un rollback par la suite





- Les **batch updates** sont une fonctionnalité ajoutée à l'interface **Statement** à partir de JDBC 2.0 pour permettre le regroupement de plusieurs traitements qui seront envoyés en une seule fois au SGBD. Elles ne sont pas nécessairement prises en charge par tous les pilotes JDBC.
- Le processus d'utilisation des batch updates avec JDBC est le suivant :
 - supportsBatchUpdates() de DatabaseMetaData : Permet de vérifier si les batch updates sont supportées par le pilote JDBC sous-jacent.
 - void addBatch(String) : Ajoute une requête SQL au batch. Il est possible de l'invoquer à partir d'un objet Statement ou PreparedStatement (sans paramètre).
 - int[] executeBatch() : Exécute toutes les requêtes SQL ajoutées au batch et retourne un tableau d'entiers représentant le résultat de chaque requête :





Le tableau d'entiers retourné contient :

- le nombre de mises à jour effectuées pour chaque requête qui a réussi (entier ≥ 0).
- `SUCCESS_NO_INFO` si la commande a été exécutée mais que le nombre de lignes affectées est inconnu.
- `EXECUTE_FAILED` si la commande a échoué.

En cas d'échec sur l'une des requêtes dans le batch, une **BatchUpdateException** est lancée.

- Il est important de noter que le comportement exact des batch updates peut varier selon les pilotes JDBC. Certains pilotes peuvent exécuter toutes les requêtes dans le batch même si une requête échoue, tandis que d'autres peuvent arrêter l'exécution du batch à la première requête qui échoue.
- `void clearBatch()` : Supprime toutes les requêtes SQL stockées dans le batch.



Exemple :

Batch update statique

```
con.setAutoCommit(false);

Statement stmt = con.createStatement();

stmt.addBatch("INSERT INTO COFFEES " +
    "VALUES('Amaretto', 49, 9.99, 0, 0)");
stmt.addBatch("INSERT INTO COFFEES " +
    "VALUES('Hazelnut', 49, 9.99, 0, 0)");
stmt.addBatch("INSERT INTO COFFEES " +
    "VALUES('Amaretto_decaf', 49,
    10.99, 0, 0)");
stmt.addBatch("INSERT INTO COFFEES " +
    "VALUES('Hazelnut_decaf', 49,
    10.99, 0, 0)");

int [] updateCounts = stmt.executeBatch();

con.commit();
con.setAutoCommit(true);
```

Batch update paramétré

```
con.setAutoCommit(false);
PreparedStatement pstmt = con.prepareStatement(
    "INSERT INTO COFFEES VALUES(?, ?, ?, ?, ?)");

pstmt.setString(1, "Amaretto");
pstmt.setInt(2, 49);
pstmt.setFloat(3, 9.99);
pstmt.setInt(4, 0);
pstmt.setInt(5, 0);
pstmt.addBatch();

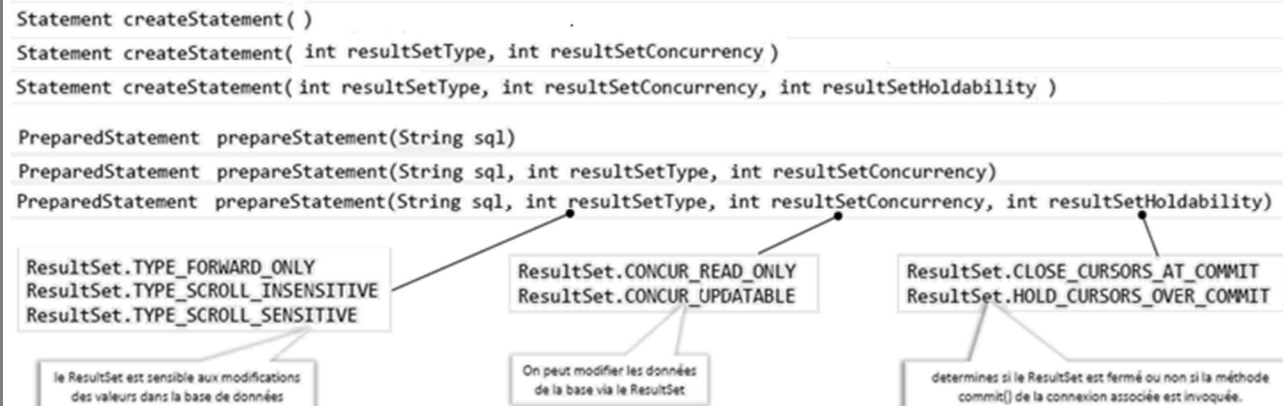
pstmt.setString(1, "Hazelnut");
pstmt.setInt(2, 49);
pstmt.setFloat(3, 9.99);
pstmt.setInt(4, 0);
pstmt.setInt(5, 0);
pstmt.addBatch();

// ... and so on for each new type of coffee

int [] updateCounts = pstmt.executeBatch();
con.commit();
con.setAutoCommit(true);
```



- Par défaut, lorsqu'on crée un **Statement** un **PreparedStatement** ou un **CallableStatement**, les objets **ResultSet** sont en **lecture seule** (read only) et à **accès séquentiel** (forward only)
- Depuis JDBC 2.0, il est possible de créer des **ResultSet** :
 - **Scrollable** : pas de limitation à un parcours séquentiel.
 - **Updatable** : possibilité de modifier les données dans la BD.





● La méthode `createStatement` (ou `prepareStatement`) dans JDBC permet de créer un objet `Statement` (ou `PreparedStatement`) à partir d'une connexion à la base de données. Cette méthode a plusieurs surcharges :

- Paramètre **`resultSetType`** : Il spécifie le type de `ResultSet`. Il peut prendre l'une des valeurs suivantes :
 - `ResultSet.TYPE_FORWARD_ONLY` : Indique un `ResultSet` à accès séquentiel uniquement. C'est la valeur par défaut. Il est possible de se déplacer dans le `ResultSet` dans un seul sens, de la première ligne à la dernière.
 - `ResultSet.TYPE_SCROLL_INSENSITIVE` : Indique un `ResultSet` scrollable mais insensible aux modifications externes. Il est possible de se déplacer dans le `ResultSet` dans les deux sens, mais les modifications apportées à la base de données par d'autres utilisateurs ne sont pas reflétées dans ce `ResultSet`.

5.12 ResultSet JDBC 2.0



- **ResultSet.TYPE_SCROLL_SENSITIVE** : Indique un ResultSet scrollable et sensible aux modifications externes. Il est possible de se déplacer dans le ResultSet dans les deux sens, et les modifications apportées à la base de données par d'autres utilisateurs sont reflétées dans ce ResultSet.
- Paramètre **resultSetConcurrency** : Il spécifie le niveau de concurrence souhaité pour le ResultSet. Il peut prendre l'une des valeurs suivantes :
 - **ResultSet.CONCUR_READ_ONLY** : Indique un ResultSet en lecture seule. On ne peut pas modifier les données de la base de données en utilisant ce ResultSet. C'est la valeur par défaut.
 - **ResultSet.CONCUR_UPDATABLE** : Indique un ResultSet modifiable. On peut modifier les données de la base de données en utilisant ce ResultSet, en utilisant des méthodes comme `updateXXX()` et `updateRow()`.

5.12 ResultSet JDBC 2.0



- Paramètre **resultSetHoldability** : Il spécifie le comportement de maintien du ResultSet lors de la fermeture de la connexion associée à un Statement. Il peut prendre l'une des valeurs suivantes :
 - **ResultSet.CLOSE_CURSORS_AT_COMMIT** : Indique que le ResultSet doit être fermé automatiquement lors de la validation (commit) de la transaction. Cela signifie que les résultats ne sont plus disponibles après le commit, même si le ResultSet est encore ouvert lors de la fermeture de la connexion. C'est la valeur par défaut.
 - **ResultSet.HOLD_CURSORS_OVER_COMMIT** : Indique que le ResultSet doit être maintenu ouvert même après la fermeture de la connexion. Cela signifie que les résultats restent disponibles après la transaction commit ou rollback, et on peut continuer à parcourir et à lire les données même après la fermeture de la connexion.



● Méthodes de parcours :

- first() : Positionne le curseur sur le premier enregistrement.
- last() : Positionne le curseur sur le dernier enregistrement.
- next() : Avance le curseur d'un enregistrement.
- previous() : Recule le curseur d'un enregistrement.
- beforeFirst() : positionne le curseur avant le premier enregistrement.
- afterLast() : Positionne le curseur après le dernier enregistrement.
- absolute(int n) : Positionne le curseur sur l'enregistrement de rang n si il existe.
- relative(int n) : Déplace le curseur de n enregistrements par rapport à sa position courante.





- Ces méthodes renvoient un booléen : true si le curseur désigne un enregistrement, false sinon
- **Méthodes de test de la position du curseur :**
 - boolean isFirst() : true si curseur est positionné sur la première ligne.
 - boolean isBeforeFirst() : true si curseur est positionné avant la première ligne.
 - boolean isLast() : true si curseur est positionné sur la dernière ligne.
 - boolean isAfterLast() : true si curseur est positionné après la dernière ligne
 - int getRow() : Renvoie le rang de l'enregistrement (le numéro de la ligne) sur lequel le curseur est positionné.



● Modification du ResultSet

- Se placer sur l'enregistrement concerné
- Méthodes updateXXX(...) où XXX désigne le type Java
- Puis updateRow() pour valider les modifications (enregistrement dans la BD)
 - le faire avant de déplacer le curseur sur une autre ligne (sinon les modifications seront ignorées)

```
Statement stmt = con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                                     ResultSet.CONCUR_UPDATABLE);
ResultSet rs = stmt.executeQuery("SELECT ID_PRODUIT,NOM,PRIX,QTE FROM STOCKS);

...
if (rs.absolute(10)) { // positionnement du curseur sur l'enregistrement

    rs.updateDouble(3, 99.99); // modification du champ PRIX
    rs.updateInt("QTE",1000); // modification du champ QTE

    rs.updateRow(); //
}
```





● Insertion d'une ligne

- moveToInsertRow()
- Méthodes updateXXX(...)
- Puis insertRow()

```
ResultSet rs = stmt.executeQuery("SELECT NOM,ID_CLIENT FROM CLIENTS");  
rs.moveToInsertRow();  
rs.updateString(1,"Jacques OUILLE");  
rs.updateInt(2,151970);  
rs.insertRow();
```

● **moveToCurrentRow()** permet de se repositionner sur la ligne courante.

● Suppression d'une ligne

- Se placer sur la ligne
- deleteRow()

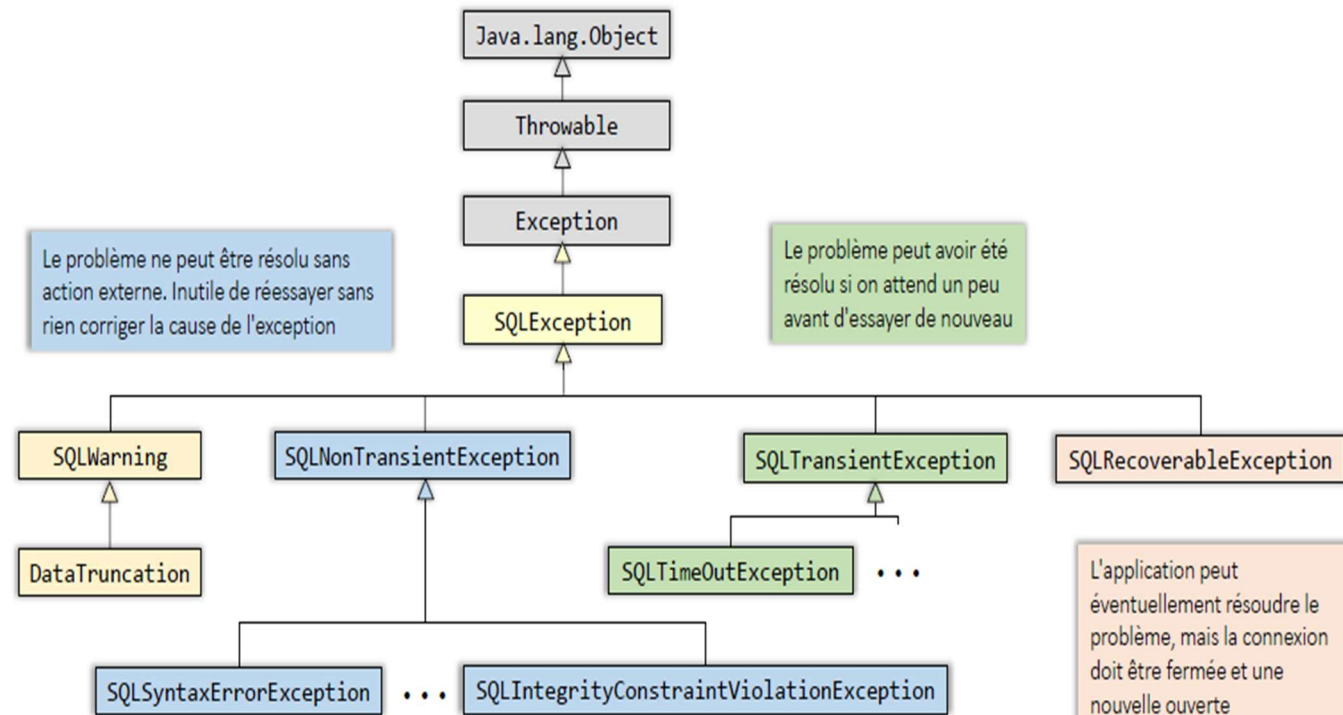
```
rs.last();  
rs.deleteRow();
```



- Tous les **ResultSet** ne sont pas nécessairement modifiables
- Tous les drivers JDBC ne supportent pas nécessairement et entièrement les **ResultSet** « scrollable » et « updatable »
 - L'objet **DataBaseMetaData** fournit des informations quant au support proposé pour les **ResultSet**
 - boolean `supportsResultSetType(int type)`
 - boolean `supportsResultSetConcurrency(int type, int concurrency)`
 - boolean `supportsResultSetHoldability(int holdability)`
 - Il est important d'être prudent si le programme que l'on écrit doit interagir avec une grande variété de drivers JDBC.

5.13 A propos de SQLException

● Les exceptions dans API JDBC 4.0



5.13 A propos de SQLException



● SQLException définit les méthodes suivantes :

- `getSQLState()` : Retourne une chaîne de caractères représentant le « SQLState » de l'erreur. Le SQLState est un code standardisé défini par le SQL standard qui permet d'identifier le type d'erreur survenue.
- `getErrorCode()` : Retourne un entier représentant le code de l'erreur. Ce code est spécifique à chaque Driver.
- `getNextException()` : Permet aux classes du JDBC de chaîner une suite de SQLException

```
try {  
    ...  
}  
catch (SQLException e) {  
    while (e != null) {  
        System.out.println("SQL Exception");  
        System.out.println(e.getMessage());  
        System.out.println("ANSI-92 SQL State : "+e.getSQLState());  
        System.out.println("Vendor error code : "+e.getErrorCode());  
        e = e.getNextException();  
    }  
}
```



5.13 A propos de SQLException

- JDBC peut générer des **avertissements** (SQLWarning) sans lancer des exceptions lorsqu'un problème est survenu mais qu'il n'est pas assez grave pour interrompre le programme.
- **SQLWarning** encapsule des avertissements qui sont générés lors des opérations JDBC et contient généralement les mêmes informations que SQLException
 - Pour récupérer ces avertissements : pas de bloc try catch mais à l'aide de la méthode **getWarnings()** des interfaces Connection, ResultSet, Statement, PreparedStatement et CallableStatement.

```
void printWarnings(SQLWarning warn) {
    while (warn != null) {
        System.out.println("\nSQL Warning");
        System.out.println(warn.getMessage());
        System.out.println("ANSI-92 SQL State : "+warn.getSQLState());
        System.out.println("Vendor error code : "+warn.getErrorCode());
        warn = warn.getNextException();
    }
}
```

```
...
ResultSet rs = stmt.executeQuery("SELECT * FROM CLIENTS");
printWarnings( stmt.getWarnings() );
printWarnings( rs.getWarnings() );
...
```